GenAI Fundamentals 🧠

Understanding How Generative AI Actually Works

Session 3 — Python-AI Series

Traditional AI vs Generative AI 🔄

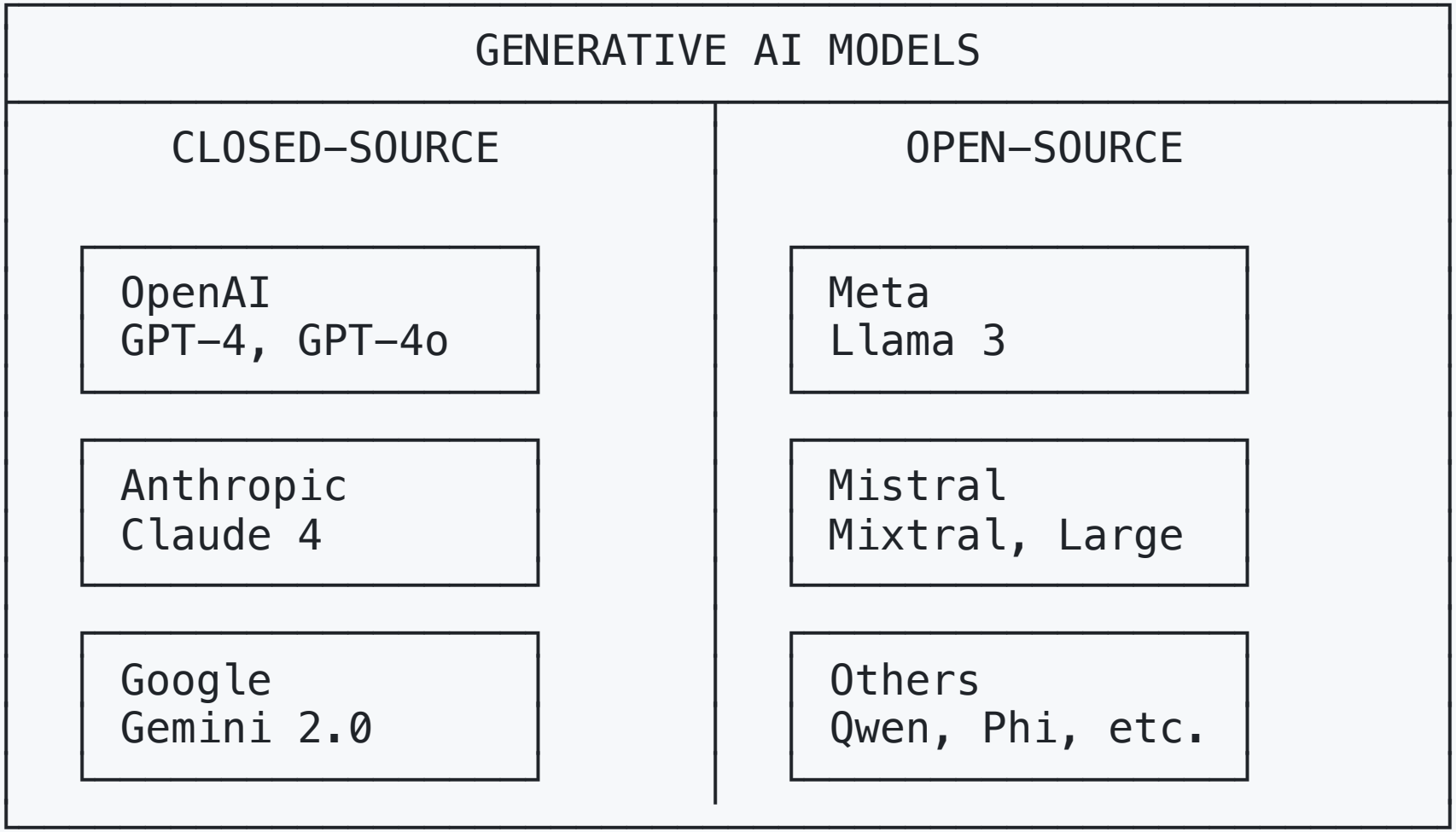
What's the Difference?

Traditional AI/ML	Generative AI
Classifies existing data	**Creates** new content
"Is this email spam?"	"Write me an email"
"What's the stock price?"	"Summarize this report"
Finds **patterns**	Generates **new outputs**

Key Insight

- 🏷️ Traditional AI = **labeling & predicting** from known categories
- ✨ Generative AI = **creating** text, images, code, music that never existed

The GenAI Landscape



Why Now? Three Breakthroughs 🚀

```
   2017                      2020                      2022–2024
    |                         |                          |
    ▼                         ▼                          ▼
 Transformers              Scaling Laws               RLHF

 "Attention                More data +                Human feedback
  is all you      ──►      more compute      ──►      makes models
  need"                     = better                  helpful & safe
  (Google)                  models
```

🧩 Better architecture + 📈 More scale + 👩‍🏫 Human alignment = **ChatGPT, Claude, etc.**

How LLMs Work ⚙️

What's actually happening inside these models?

Tokens: The Atoms of Language 🔤

LLMs don't read words — they read tokens

```
Input:  "The cat sat on the mat"
Tokens:
         ┌─────┐  ┌─────┐  ┌─────┐  ┌─────┐  ┌─────┐  ┌─────┐
         │ The │  │ cat │  │ sat │  │ on  │  │ the │  │ mat │
         └─────┘  └─────┘  └─────┘  └─────┘  └─────┘  └─────┘
            ▼        ▼        ▼        ▼        ▼        ▼
IDs:     [ 464 ] [3797 ] [3332 ] [ 319 ] [ 262 ] [2603 ]
```

```
"Unbelievable!" → ["Un", "believ", "able", "!"]   (4 tokens)
"AI"            → ["AI"]                           (1 token)
"anthropic"     → ["anthrop", "ic"]                (2 tokens)
```

- 📏 **Rule of thumb:** 1 token ≈ 4 characters ≈ ¾ of a word
- 🔢 The model ONLY sees numbers — everything is math!

The Prediction Machine 🎰

An LLM does ONE thing: predict the next token

```
Input: "The cat sat on the"

┌──────────────────────┐
│  Next token odds:    │
│                      │
│  "mat"     → 32%     │
│  "floor"   → 18%     │
│  "couch"   → 12%     │
│  "bed"     →  8%     │
│  "table"   →  6%     │
│  ...rest   → 24%     │
└──────────────────────┘
```

How it writes a full response

```
"What is Python?"  → "Python"
"...? Python"      → "is"
"...Python is"     → "a"
"...is a"          → "programming"
"...a programming" → "language"
```

- 🔁 One token at a time, fed back as input
- 🧠 No "understanding" — just very sophisticated pattern matching

Training Pipeline - Three Stages to Build an LLM 🏋️

PRE-TRAINING	FINE-TUNING	RLHF
Read the entire internet Data: Books, web, Wikipedia, code repos Result: Knows language & facts Cost: $$$$$	Learn to be a helpful assistant Data: Q&A pairs, instruction examples Result: Follows instructions Cost: $$$	Learn human preferences Data: Human rankings of "A vs B" responses Result: Helpful & safe Cost: $$

Context Window = Working Memory 📋

Everything must fit on the "whiteboard"

```
┌──────────────────────────────────────────────────────────────────────┐
│                  CONTEXT WINDOW (e.g., 128K tokens)                  │
│                                                                      │
│   ┌──────────────────┐   ┌──────────────────────────┐   ┌──────────────────────┐
│   │ System           │   │ Conversation             │   │ Model's              │
│   │ Prompt           │   │ History                  │   │ Response             │
│   │ (rules)          │   │ (all messages)           │   │ (generating)         │
│   └──────────────────┘   └──────────────────────────┘   └──────────────────────┘
│                                                                      │
│  ◄──────── EVERYTHING must fit in here ──────────────────────────►  │
└──────────────────────────────────────────────────────────────────────┘
```

📌 It is like how we use a whiteboard — once full, older stuff starts getting erased :(

Temperature: The Creativity Dial 🌡️

Controls how "random" the model's choices are

```
Temperature = 0 (Precise)                Temperature = 1.0 (Creative)
──────────────────────────                ────────────────────────────

  90% ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇                  25% ▇▇▇▇▇▇▇
   5% ▇▇                                  22% ▇▇▇▇▇▇▇
   3% ▇                                   20% ▇▇▇▇▇▇
   2% ▇                                   18% ▇▇▇▇▇▇
                                          15% ▇▇▇▇▇

→ Always picks top choice                 → Any top option could win
```

When to Use What

Temperature	Use Case
0.0	Factual Q&A, code, math
0.5	Balanced chat applications
0.7	Creative writing, brainstorming
1.0+	Poetry, wild ideas (may be incoherent)

Section 2 Recap ✅

How LLMs Work — Key Takeaways

- 🔤 **Tokens** — text split into pieces, converted to numbers
- 🎰 **Prediction** — model predicts one token at a time
- 🏋️ **Training** — pre-training → fine-tuning → RLHF
- 📋 **Context window** — all input + output must fit in memory
- 🌡️ **Temperature** — dial between precise and creative

The Big Insight

> LLMs are **pattern completion engines** trained on human text.
> No understanding — just incredibly good at predicting what comes next.

Limitations, Risks & Realities ⚠️

Why LLMs alone aren't enough

The Three Technical Limitations 🚧

HALLUCINATIONS	KNOWLEDGE CUTOFF	NO PRIVATE DATA
Confidently states things that are FALSE. "The Eiffel Tower is 500m" (it's 330m)	Training data has a date. Nothing after that exists. "Who won the 2026 World Cup?" → "I don't know"	Can't access YOUR company's documents, DBs, or wikis. "What's our refund policy?" → "I don't know"

Responsible AI Considerations 🛡️

BIAS IN DATA	DATA PRIVACY	HUMAN OVERSIGHT
Models learn from internet text which contains societal biases. Outputs can reflect & amplify these.	Sending data to an API = sending it to a third party. Ask: Is this sensitive? Is it compliant?	AI assists, humans decide. Critical decisions MUST have human review. (hiring, medical, legal)

Before deploying AI, ask:

- ❓ What data am I sending? Is it sensitive/confidential?
- ❓ Could bias in outputs affect people unfairly?
- ❓ Who reviews the AI's output before it reaches users?

Cost Awareness & The Knowledge Gap 💰

API Costs Add Up

Operation	Approximate Cost
GPT-4 / Claude input	~$2-5 per 1M tokens
Embedding 1M tokens	~$0.10
Vector DB storage	Pennies per 1K docs
Fine-tuning	$100s-$1000s per run

> 💡 A chatbot with 10K queries/day can cost $100+/day in API calls!

The Solution to the Knowledge Gap

```
What LLM knows:          GAP          What you need:
                      ←──────────→
┌──────────────────┐                  ┌──────────────────┐
│ General world    │                  │ Your company     │
│ knowledge up     │   🌉 RAG bridges  │ docs, data,      │
│ to cutoff        │     this gap!    │ private info     │
└──────────────────┘                  └──────────────────┘
```

Embeddings & Vector Search

How machines understand meaning

What Are Embeddings? 🔢

Text → Numbers that capture MEANING

```
Text                          Embedding (simplified — usually 768–1536 numbers)
──────────────────────────    ──────────────────────────────────────────────────
"king"                  →     [0.21, 0.83, -0.45, 0.67, 0.12, ...]
"queen"                 →     [0.19, 0.81, -0.42, 0.71, 0.14, ...]  ← similar!
"banana"                →     [0.92, -0.31, 0.55, -0.12, 0.88, ...]  ← different!
```

The Key Insight

- ✅ **Similar meanings → similar numbers**
- ✅ "King" and "queen" have nearby vectors
- ❌ "Banana" is far away in number-space
- 🌐 Works across languages and phrasings!

Semantic Space — Meaning Has a Map 🗺️

```
              SEMANTIC SPACE (reduced to 2D)
     ┌──────────────────────────────────────────────────────┐
     │              ● "happy"                               │
     │          ● "joyful"        ● "excited"               │
     │            ● "cheerful"                              │
     │                                   ● "sad"            │
     │                                ● "unhappy"           │
     │                              ● "depressed"           │
     │  ● "python"                                          │
     │    ● "javascript"                                    │
     │      ● "coding"                                      │
     │                                                      │
     └──────────────────────────────────────────────────────┘
```

- 🎯 Similar concepts **cluster together** regardless of spelling!
- 🔍 This is why "forgot my login" finds "password reset help"

Cosine Similarity — Measuring "How Similar?" 📐

Think of embeddings as arrows

```
Same direction          Perpendicular          Opposite
= Very similar          = Unrelated            = Opposite

   ↗                          ↑                   ↗
  ↗                        →                    ↙

Score ≈ 1.0             Score ≈ 0.0            Score ≈ -1.0
```

Real Examples

Sentence A	Sentence B	Score
"How do I reset my password?"	"I forgot my login credentials"	**0.92**
"How do I reset my password?"	"Steps to recover account access"	**0.87**
"How do I reset my password?"	"What's the weather today?"	**0.13**
"The cat sat on the mat"	"A feline rested on the rug"	**0.89**

Vector Databases — Semantic Search Engines 🔎

Keyword Search vs Vector Search

```
KEYWORD SEARCH                          VECTOR SEARCH
──────────────────────                  ──────────────────────
Query: "password reset"                 Query: "I can't log in"
Looks for EXACT WORDS                   Finds MEANING matches
❌ Misses: "account recovery"           ✅ Finds: "password reset guide"
❌ Misses: "login help"                 ✅ Finds: "account recovery steps"
```

Popular Vector Databases

- 🌲 **Pinecone** — fully managed, easy to start
- 🔷 **ChromaDB** — open-source, great for prototyping
- 🐘 **pgvector** — PostgreSQL extension (use your existing DB)
- ⚡ **FAISS** — Facebook's library, blazing fast

Chunking & Document Processing

Breaking documents into searchable pieces

Why Chunk? ✂️

You can't embed a 100-page document as one vector

```
┌──────────────────────────────────────────────────────────┐
│                   100-PAGE DOCUMENT                      │
│  One embedding for ALL of this?                          │
│  → Meaning is too vague/diluted                          │
│  → Can't pinpoint which PART is relevant                 │
│  → Too big to fit in LLM context anyway                  │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼ CHUNK IT

┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│Chunk 1   │  │Chunk 2   │  │Chunk 3   │  │Chunk 4   │  │Chunk 5   │
│~500 tok  │  │~500 tok  │  │~500 tok  │  │~500 tok  │  │~500 tok  │
│Own       │  │Own       │  │Own       │  │Own       │  │Own       │
│embed.    │  │embed.    │  │embed.    │  │embed.    │  │embed.    │
└──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘
```

- ✅ Each chunk has **focused meaning**
- ✅ Each can be **retrieved independently**
- ✅ Each **fits** in LLM context window

Chunking Strategies 🧩

```
STRATEGY 1: Fixed-size
──────────────────────────
[  500 tokens  ][  500 tokens  ][  500 tokens  ]
                ↑
    ⚠️ Might split mid-sentence!
✅ Simple    ❌ Can break meaning
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
STRATEGY 2: Sentence/paragraph-based
──────────────────────────────────────
[Paragraph 1][  Paragraph 2  ][Para 3][Paragraph 4  ]

✅ Respects natural breaks    ❌ Uneven sizes
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
STRATEGY 3: Semantic chunking
──────────────────────────────────────
[ Topic A content ][ Topic B content ][ Topic C ]

✅ Best quality    ❌ More complex to implement
```

Overlap: Don't Lose Context 🔗

```
WITHOUT overlap:
──────────────────────────────────────────────────────────
...the refund policy requires │ customers to submit within 30 days...
         Chunk 1 ends here ───┘└─── Chunk 2 starts here

⚠️ Neither chunk has the FULL refund policy!


WITH overlap (shared content):
──────────────────────────────────────────────────────────
Chunk 1: ...the refund policy requires customers to submit within 30 days...
Chunk 2:       ...policy requires customers to submit within 30 days of purchase...
                ↑──────────────── Overlapping region ──────────────↑

✅ Both chunks have the complete information!
```

- 📏 **Typical overlap:** 10-20% of chunk size
- 💡 e.g., 500-token chunks with 50-100 token overlap

RAG: Retrieval-Augmented Generation 🔄

The solution that makes LLMs actually useful for YOUR data

What is RAG? 📖

```
WITHOUT RAG:                          WITH RAG:
────────────────────                  ────────────────────
You: "What's our                      You: "What's our
      refund policy?"                       refund policy?"
                                                  │
                                                  ▼
                                      [🔍 Search your docs]
                                                  │
                                                  ▼
                                      [Found: policy.pdf section 4.2]
                                                  │
                                                  ▼
LLM: "I don't have                    LLM: "Based on your policy, refunds
      that information"                     are available within 30 days
                                            with receipt. See section 4.2"
```

📚 It's like an open-book exam — hand the student the right pages, THEN ask

The Full RAG Pipeline 🏗️

Phase 1: Ingestion (one-time setup)

```
Your Documents  ──►  CHUNK into pieces  ──►  EMBED each chunk  ──►  STORE in Vector DB
```

Phase 2: Query (every question)

```
User Question  ──►  EMBED query  ──►  SEARCH Vector DB  ──►  RETRIEVE Top 3–5
                                                                  │
                                                                  ▼
                                                  Send chunks + question
                                                  to LLM → Get answer
```

RAG in Pseudocode

```
# ═══ PHASE 1: INGESTION (run once) ═══

documents = load_documents("company_docs/")
chunks = split_into_chunks(documents, size=500, overlap=50)
embeddings = embed(chunks)
vector_db.store(chunks, embeddings)

# ═══ PHASE 2: QUERY (every question) ═══

question = "What is our refund policy?"
question_embedding = embed(question)
relevant_chunks = vector_db.search(question_embedding, top_k=3)

prompt = f"""
Using ONLY this context, answer the question.
If not in context, say "I don't know."

Context: {relevant_chunks}
Question: {question}
"""
answer = llm.generate(prompt)
```

What is Fine-tuning? 🎓

Teaching the model NEW behavior with YOUR data

```
Fine-tuning = take a pre-trained model and train it further on your specific data

┌──────────────────────┐   +   ┌──────────────────────┐   =   ┌──────────────────────┐
│   Base Model         │       │   Your Data          │       │  Specialized         │
│   (GPT, Claude,      │       │   (examples of       │       │  Model               │
│    Llama, etc.)      │       │   desired input/     │       │  (behaves the        │
│                      │       │   output pairs)      │       │   way you want)      │
└──────────────────────┘       └──────────────────────┘       └──────────────────────┘
```

- 👩‍🍳 Like hiring a trained chef (base model) and teaching them YOUR restaurant's recipes (fine-tuning)
- The chef already knows how to cook — you're just specializing their skills
- 🎭 Custom tone/style (e.g., "write like our brand voice")
- 🏥 Domain-specific language (medical, legal, financial)

RAG vs Fine-tuning ⚖️

Factor	RAG ✅	Fine-tuning
Data freshness	Always current (update docs anytime)	❌ Frozen at training time
Cost	Cheap (storage + search)	❌ Expensive GPU training
Hallucination	Can cite sources	❌ May still make things up
Setup time	Hours	❌ Days to weeks
Best for	Facts, docs, knowledge	Style, tone, behavior

The Rule of Thumb

- 📚 Need the model to **know** something? → **RAG**
- 🎭 Need the model to **behave** differently? → **Fine-tuning**
- 💡 Most real-world use cases → **RAG**

Prompt Engineering Essentials 💬

How to talk to LLMs effectively

System Prompts: Setting the Stage 🎬

Tell the model WHO it is and HOW to behave

```
┌─ System Prompt ──────────────────────────────────────────────┐
│
│  You are a customer service agent for Acme Corp.
│  - Be concise and professional
│  - Always cite the relevant policy number
│  - If you don't know, say "I don't know"
│  - Never make up information
└──────────────────────────────────────────────────────────────┘
          │
          ▼ Shapes ALL responses in the conversation
```

- 🎯 Sets boundaries and expectations
- 🛡️ Prevents unwanted behavior
- 📋 Consistent tone and format across responses

Few-Shot: Teaching by Example 📝

```
PROMPT:
────────────────────────────────────────
Convert informal text to professional email language.

Input: "Hey, can u fix this ASAP?"
Output: "Hello, could you please address this at your earliest convenience?"
Input: "thx for the help!"
Output: "Thank you for your assistance."
Input: "gonna need that report by tmrw"
Output: ???
────────────────────────────────────────
MODEL RESPONSE:
"I will need that report by tomorrow."
```

- 🧠 It learned the pattern from just **2 examples**!
- 📈 More examples = more consistent results

Chain-of-Thought: "Think Step by Step" 🧮

Forces the model to show its work

```
WITHOUT CoT:                          WITH CoT:
────────────────                      ────────────────

Q: "3 shirts at $25 each,            Q: "Think step by step.
    20% off. I have $60.                  3 shirts at $25 each,
    Can I buy all 3?"                     20% off. I have $60.
                                          Can I buy all 3?"
A: "Yes"                              A: "Step 1: Price = $25 each
    ← sometimes wrong!                    Step 2: 20% off = $5 discount
                                          Step 3: Sale price = $20
                                          Step 4: 3 × $20 = $60
                                          Step 5: $60 = my budget
                                          Yes! Exactly enough." ✅
```

🎯 Dramatically improves accuracy on reasoning tasks

Recap 🎉

What We Covered Today

- 🔤 **Tokens** — text becomes numbers
- 🎰 **LLMs** — next-token prediction machines
- 📋 **Context window** — model's working memory
- 🧲 **Embeddings** — "meaning fingerprints" for text
- 🔎 **Vector search** — find by meaning, not keywords
- ✂️ **Chunking** — break docs into searchable pieces
- 🔄 **RAG** — retrieve → augment → generate
- 🎓 **Fine-tuning** — specialize a model with your data
- 💬 **Prompt engineering** — system prompts, few-shot, CoT

Quick Reference Card 📇

Term	Meaning
Token	A piece of a word (~4 characters)
LLM	Predicts next token based on patterns
Context Window	How much text model sees at once
Temperature	Creativity dial (0=precise, 1=creative)
Embedding	Text → numbers capturing meaning
Vector DB	Database for similarity search
Chunking	Splitting docs into small pieces
RAG	Retrieve docs, then generate answer
Fine-tuning	Retrain a model on your specific data